Deliverable 2.2

Scenario

RocketTickers expects 50,000 users in the first 90 seconds.
Flow:

1. Inventory (atomic)
2. Payment (800ms, can fail)
3. Ticket
4. Notification
5. Audit

Team Alpha (Inventory + Payment)

- Wants a Central *PurchaseOrchestrator*
- Prioritises explicit rollback after past “charged but no ticket” failure

Team Beta (Ticket + Notification + Audit)

- Wants choreography on PaymentConfirmed
- Prioritises scale and avoiding a flash-sale bottleneck

What we need to answer:
Which pattern do you recommend, and why?
What is the single biggest risk, and how do you mitigate it?
What would change your recommendation?

CTO is about to start. We are the advisory panel between the two teams.

Full Orchestration

A single `PurchaseOrchestrator` drives all five steps sequentially, with explicit compensation at every failure point.

Advantages

- Full visibility into purchase state at any moment
- Compensations are explicit and testable (release inventory, refund payment)
- Directly addresses the "charged but no ticket" failure mode Team Alpha experienced
- Easier to debug when something goes wrong

Disadvantages

- The orchestrator becomes the flash-sale bottleneck; at 555 req/s with an 800ms payment step, queue depth explodes within seconds

- A single orchestrator crash mid-flow leaves purchases in an unknown state
- Notification and Audit have no business being in the critical path, but here they are
- Horizontal scaling the orchestrator introduces distributed state coordination, which erases the simplicity argument

Full Choreography - seloko mano n compensa

Services react to domain events. `InventoryReserved` triggers payment, `PaymentConfirmed` triggers ticket creation, and so on.

Advantages

- Each service scales independently; no flash-sale bottleneck
- Teams own their slice completely
- Natural fit for Notification and Audit, which are genuinely fire-and-forget

Disadvantages

- The "charged but no ticket" failure is *harder* to detect, not easier. The payment service emits `PaymentConfirmed` and walks away. If the ticket service crashes, nobody is watching
- Saga rollback becomes implicit and event-driven compensation is significantly harder to reason about
- Tracing a single purchase across five services under load is painful
- Team Alpha's trust problem is not addressed; it's redistributed

Hybrid Saga

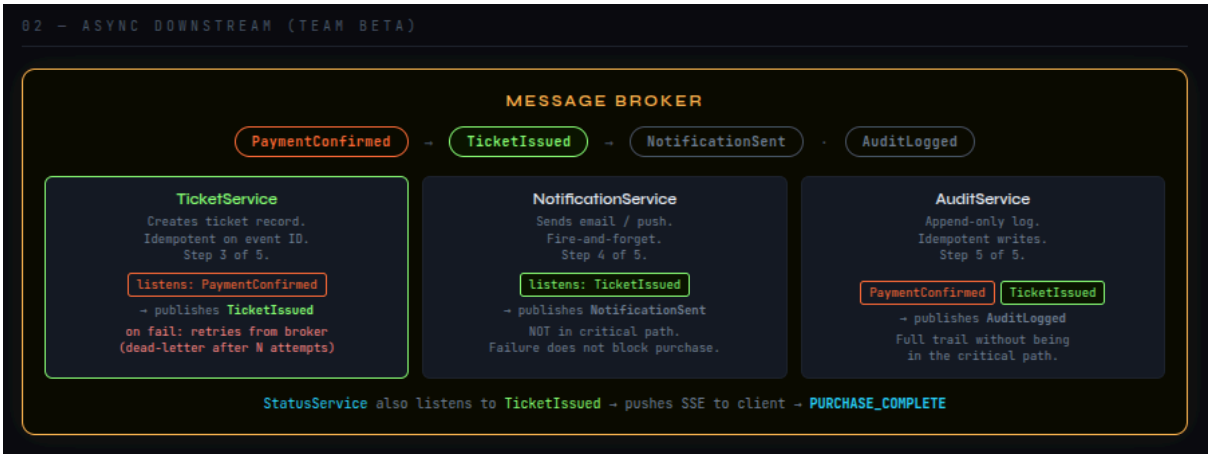
Split the flow at the natural trust boundary.

Orchestrated critical path (steps 1 and 2): A `PurchaseOrchestrator` owns Inventory and Payment exclusively. It reserves inventory, calls payment, and if payment fails, it explicitly releases the reservation. This is a two-step saga with one compensation. It is small, testable, and directly solves the "charged but no ticket" precondition. The orchestrator's only job is to produce a trustworthy `PaymentConfirmed` event or roll back cleanly.

Choreographed downstream (steps 3, 4, 5): On `PaymentConfirmed`, Ticket, Notification, and Audit react independently via a message broker. They are idempotent consumers. If Ticket creation fails, it retries from the event log. Notification and Audit are entirely decoupled from purchase latency.

```
User Request
   |
[PurchaseOrchestrator]
   |--> Inventory.Reserve()   (atomic, fast)
```

|→ Payment.Charge() (800ms, can fail)
| |-- FAIL → Inventory.Release() + return error
| |-- OK → publish PaymentConfirmed
| (SPF)
[Message Broker]
  |-- TicketService (creates ticket, publishes TicketIssued)
  |-- NotificationService (reacts to TicketIssued)
  |-- AuditService (reacts to PaymentConfirmed)



Which pattern do you recommend, and why?

Hybrid Saga. The orchestrator owns Inventory and Payment exclusively — the two steps where money and stock are at risk. It produces one durable event (`PaymentConfirmed`) and its job is done. Everything after that is choreography on the broker. This gives Team Alpha the explicit rollback they need after their "charged but no ticket" incident, and gives Team Beta the independent scaling they need for the flash sale. Neither team compromises their core requirement.

The key argument: the two teams are not in conflict. They are describing two different parts of the same system. The pattern reflects that.

What is the single biggest risk, and how do you mitigate it?

The orchestrator commits the payment and then fails before publishing `PaymentConfirmed`. Money taken, nothing downstream fires. This is not a bug in the code — it is a fundamental property of writing to two systems (database and broker) without atomicity.

Mitigation is the outbox pattern. The orchestrator writes `PaymentConfirmed` to an outbox table in the same database transaction as the payment record. A relay process publishes it to the broker separately. The event cannot be lost because it lives in your own database until the broker acknowledges it. Downstream consumers must be idempotent to handle the at-least-once delivery this introduces.

What would change your recommendation?

Four things could flip it:

The team has no time to build outbox infrastructure. The hybrid introduces a failure mode they cannot handle. Fall back to full orchestration with Notification and Audit moved to an async queue.

Payment latency drops below 100ms. At that point full orchestration becomes viable under flash-sale load and the simpler mental model wins.

Ticket issuance has hard consistency requirements — seat assignment, sequential numbering, PDF generation. Pull it back into the orchestrated path rather than trusting choreography to handle it correctly.

The team's operational maturity is low. Debugging a choreographed flow across five services under 50k users in the first 90 seconds of a sale will cause more incidents than the scale benefit is worth. Simpler to operate beats theoretically optimal.

Nem tu nem eu

Telemetry Hiding/Blocking of Information

Keep

- User Id
- Seat
- Ticket Id
- Transaction ID + State
- Outcome
- Payment Provider
- Request Route, Status Code, Latency
- Service Name + Operation

Redacted

- Card Number

Block

- Email
- Phone
- Raw Bodies of Request/Response
- User Id as a metric label/tag